
ARCA: Adapter-Residual Credit Assignment When Token Signals Degenerate

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Token-level credit assignment for language-model reinforcement learning is usually
2 formulated as if the policy were fully trainable, while practical LLM-RL pipelines
3 overwhelmingly rely on parameter-efficient fine-tuning, especially LoRA. We ar-
4 gue that this separation hides a structural failure mode. Under LoRA, the policy
5 is restricted to a low-rank neighborhood of the reference model, so the per-token
6 output-distribution differences used by common intrinsic credit signals—surprisal,
7 entropy reduction, and policy divergence—can be driven toward degenerate nor-
8 malized weights. We formalize this degeneration and propose measuring it directly
9 with concentration diagnostics such as weight Gini and effective-token ratio. We
10 then introduce *Adapter-Residual Credit Assignment* (ARCA), a lightweight alter-
11 native that derives token salience from the adapter’s own hidden-state residual,
12 $\|h_t^{\text{adapted}} - h_t^{\text{base}}\|_2$. ARCA asks where the adapter actually changes the model,
13 rather than where the output distribution appears uncertain or shifted, and re-
14 quires no learned reward model, value head, or tree construction. In a compact
15 MATH/Qwen3-1.7B GRPO sweep, ARCA exhibits the predicted non-degenerate
16 middle-regime credit distribution under matched rollout budgets and remains com-
17 petitive with rank-matched baselines.

18 1 Introduction

19 Reinforcement learning has become a central component of large language model (LLM) post-training,
20 particularly for alignment and for reasoning-oriented tasks with verifiable rewards. A persistent
21 challenge across these settings is *credit assignment*: trajectories are long, rewards are sparse and
22 outcome-level, and it is not obvious how a single scalar signal at the end of a generation should be
23 distributed across hundreds or thousands of token decisions. Recent work has responded with a rapidly
24 growing toolbox of token-level credit-assignment methods, including entropy-aware modulation,
25 process reward models, tree-based prefix values, and reward redistribution [7, 16, 32, 34, 9, 41, 14].

26 In parallel, much open-source and academic LLM-RL training uses parameter-efficient fine-tuning,
27 most commonly LoRA [11]. This is an empirical reality driven by memory and compute constraints:
28 widely used RL frameworks such as TRL and verl support LoRA, and recent results show that LoRA
29 + RL can be dramatically more sample- and parameter-efficient than full fine-tuning [33]. Yet the
30 two research threads are developed in near-total isolation. Papers on token-level credit assignment
31 specify methods without reference to the adaptation strategy, and PEFT is treated as an orthogonal
32 implementation detail.

33 In this work we argue that this separation is a mistake, and that the *interaction* between PEFT
34 and credit assignment is itself a first-class scientific object. The core issue is geometric. Intrinsic
35 token-level weighting schemes derive per-token salience from quantities such as surprisal, entropy
36 reduction, or divergence between the policy and a reference model. Under LoRA, however, the

policy is constrained to a small low-rank neighborhood of the reference, and the per-token differences that these signals measure can lose the variation that made them useful. We formalize this as degeneration of the normalized salience distribution, characterize it with Gini coefficient and effective token count, and show that the same mechanism applies across multiple output-distribution-based weighting rules. This gives a mechanistic explanation for the empirically observed failure of full-token GRPO + LoRA [15]: the weighting signal itself is degraded before training ever begins.

Rather than trying to recover per-token structure from signals that LoRA intrinsically flattens, we measure salience directly from the adapter’s own contribution to the forward pass. Adapter-Residual Credit Assignment (ARCA) sets the salience at position t equal to the norm of the adapter residual, $\|h_t^{\text{adapted}} - h_t^{\text{base}}\|_2$, computed via a forward pass with the adapter disabled. This signal is positive whenever the adapter is active, non-uniform across positions because adapter input activations are non-uniform, and requires no additional networks, learned reward models, or tree construction.

Concretely, this paper makes three contributions:

1. We identify and formalize a PEFT-specific failure mode of token credit assignment: under LoRA, output-distribution salience can degenerate into uniform broadcast or spurious sparsity, even when the underlying RL objective is unchanged.
2. We introduce ARCA, a lightweight adapter-residual credit assignment rule whose normalized token weights remain non-degenerate whenever the adapter has position-varying hidden-state impact.
3. We validate the mechanism with concentration diagnostics and a matched MATH/Qwen3-1.7B sweep, separating downstream performance from the more basic question of whether a proposed token-credit signal survives the adaptation regime actually used in LLM-RL.

The remainder of the paper develops these contributions. Section 2 positions the work relative to PEFT for language RL and token-level credit assignment. Section 3 presents the weighting schemes, explains why output-distribution signals degenerate under LoRA, and defines ARCA. Section 4 reports the diagnostic and performance comparison on MATH with Qwen3-1.7B under the seven-run submission-time sweep. Extended related work and theoretical interpretation appear in Appendices A and C.

2 Related Work

2.1 PEFT and Low-Rank Adaptation in Language RL

Parameter-efficient fine-tuning, and in particular LoRA [11], is a dominant adaptation strategy in open-source LLM-RL work. Tina demonstrated that tiny LoRA adapters on a 1.5B base model suffice to reach DeepSeek-R1-class reasoning behavior when combined with GRPO, at a tiny fraction of full fine-tuning cost [33]. A broader systematic evaluation of PEFT methods under RLVR [39] benchmarks over a dozen PEFT variants (including DoRA, AdaLoRA, MiSS, PiSSA, MiLoRA, VeRA and Rank-1) on DeepSeek-R1-Distill models and finds that standard LoRA is not optimal—structural variants such as DoRA, AdaLoRA and MiSS consistently outperform it, and SVD-initialized variants (PiSSA, MiLoRA) suffer from *spectral collapse*, a finding that is broadly compatible with our own signal-degeneration analysis. Token-Efficient RL introduces critic-free LoRA-compatible variants of GRPO (S-GRPO and T-SPMO) that focus training on a subset of tokens, reporting large gains on small models where full-token GRPO + LoRA trains unstably [15]; we now provide a mechanistic explanation for that instability via signal degeneration. *LoRA as an Implicit KL Regularizer* analyzes how LoRA restricts the policy to a rank-constrained neighborhood of the reference, deriving an explicit rank-dependent upper bound on the KL divergence between policy and reference throughout training [2]. Our Section 3.4 builds directly on this observation: an implicit KL bound is the same mechanism that forces per-token log-probability differentials to be small and small, which is the formal starting point for our degeneration result.

On the analysis side, *Narrow Fine-Tuning Traces* shows that narrow fine-tuning leaves clearly readable traces in the activation differences between base and fine-tuned hidden states, and that the fine-tuning domain can be recovered from those differences alone using simple diffing tools such as patchscopes and activation steering [21]. This is directly relevant to ARCA because the adapter residual $h_t^{\text{adapted}} - h_t^{\text{base}}$ is a per-position activation difference of exactly the kind they study; their

89 results supply empirical evidence that this difference carries meaningful, semantically non-trivial
 90 content—the property our method exploits. Concurrent work on TopLoRA studies how to concentrate
 91 LoRA capacity on a small number of high-impact tokens via token-wise input-output projections [17].
 92 To our knowledge, no prior work has characterized the interaction between LoRA and token-level
 93 credit assignment, nor proposed an adaptation-aware intrinsic weighting scheme.

94 2.2 Positioning of This Work

95 The literature makes three points clear. First, critics are not strictly required for effective LLM post-
 96 training: critic-free estimators such as RLOO, ReMax, GRPO, and GSPO are already competitive in
 97 both RLHF and RLVR [1, 19, 29, 43]. Second, many researchers have concluded that trajectory-level
 98 rewards are too coarse and have introduced denser supervision through process reward models,
 99 redistribution rules, tree structures, optimal baselines, temporal traces, or entropy-based modulation
 100 [16, 7, 32, 3, 24, 18, 13, 9, 41, 20]. Third, the overwhelming majority of practical LLM-RL pipelines
 101 are *trained with LoRA*, and a focused sub-literature has studied PEFT-specific effects in this regime [11,
 102 33, 39, 2, 15, 21, 17].

103 Our contribution is to bridge the second and third of these threads. Existing intrinsic credit-assignment
 104 signals (surprisal, entropy, divergence) are not novel as standalone objects, and we do not claim
 105 otherwise; they are our baselines. Our novel claim is that these signals *interact pathologically with the*
 106 *adaptation strategy that the field often uses*: under LoRA they can degenerate into uniform broadcast
 107 or spurious sparsity, so any paper that reports a null result for “fancy intrinsic weighting versus
 108 uniform” under LoRA may be observing a LoRA artifact, not evidence against token-level credit
 109 assignment. We formalize this, supply a unified diagnostic (Gini/EffN) that makes the degeneration
 110 visible at training time, and propose ARCA as an adaptation-aware alternative whose construction
 111 directly avoids the failure mode we identify.

112 3 Methods

113 We consider on-policy reinforcement learning for autoregressive language models with trajectory-
 114 level rewards. Let π_θ denote a language model parameterized by θ , generating a completion $y =$
 115 $(y_1, \dots, y_T)$ conditioned on a prompt x . After sampling a full trajectory, the model receives a scalar
 116 reward $R(x, y) \in \mathbb{R}$ computed by an external verifier, such as exact-answer correctness or unit-test
 117 pass rate. Our objective is

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)}[R(x, y)], \quad (1)$$

118 where $\mathcal{D}$ denotes the prompt distribution. We focus on the regime most relevant to recent reasoning-
 119 model training: sparse outcome rewards, on-policy sampling, and no learned value function.

120 3.1 Policy Gradient with Trajectory-Level Reward

121 For a fixed prompt x , the standard REINFORCE estimator is

$$g_{\text{rf}}(x, y) = R(x, y) \sum_{t=1}^T \nabla_\theta \log \pi_\theta(y_t | y_{<t}, x). \quad (2)$$

122 In practice, a baseline $b(x)$ is subtracted to reduce variance without changing the expected gradient
 123 as long as $b(x)$ does not depend on the sampled action at token t :

$$g_{\text{base}}(x, y) = (R(x, y) - b(x)) \sum_{t=1}^T \nabla_\theta \log \pi_\theta(y_t | y_{<t}, x). \quad (3)$$

124 We use prompt-level multi-sample baselines. Given K sampled completions $\{y^{(i)}\}_{i=1}^K$ for the same
 125 prompt x with rewards $\{R^{(i)}\}_{i=1}^K$:

$$b_{\text{RLOO}}^{(i)}(x) = \frac{1}{K-1} \sum_{j \neq i} R^{(j)} \quad (4)$$

126 is the leave-one-out baseline used in RLOO-style estimators, while GRPO-style normalization can be
 127 written as

$$A_{\text{GRPO}}^{(i)}(x) = \frac{R^{(i)} - \mu_R(x)}{\sigma_R(x) + \varepsilon}, \quad (5)$$

128 where $\mu_R(x)$ and $\sigma_R(x)$ are the within-prompt mean and standard deviation of the K rewards. In
 129 both cases, the resulting scalar advantage is then broadcast uniformly over all tokens in the sampled
 130 trajectory. This scalar-broadcast assumption is precisely what we relax.

131 3.2 Token-Level Credit Redistribution

132 We introduce token-level weights $w_t(x, y)$ that redistribute a trajectory-level advantage across tokens:

$$g_w(x, y) = A(x, y) \sum_{t=1}^T w_t(x, y) \nabla_{\theta} \log \pi_{\theta}(y_t \mid y_{<t}, x), \quad (6)$$

133 where $A(x, y)$ denotes either $R(x, y) - b_{\text{RLOO}}(x)$ or $A_{\text{GRPO}}(x, y)$ depending on the baseline choice.

134 The weights satisfy

$$\sum_{t=1}^T w_t(x, y) = 1, \quad w_t(x, y) \geq 0. \quad (7)$$

135 This length normalization keeps the total update magnitude comparable across weighting schemes
 136 and isolates the effect of credit redistribution from simple rescaling. It also means that the uniform
 137 baseline below is the length-normalized version of the usual token-sum estimator, rather than the
 138 unnormalized sum itself.

139 Uniform token credit corresponds to $w_t = 1/T$ for all t . Our goal is to replace this uniform allocation
 140 with *intrinsic* weights derived from quantities already produced by the policy during generation. We
 141 do not train learned token-level reward models, estimate prefix values, or build explicit search trees.

142 3.3 Intrinsic Token-Weighting Mechanisms

143 We define each method through an unnormalized salience score $\alpha_t(x, y) \geq 0$ and then normalize
 144 within each trajectory:

$$w_t(x, y) = \frac{\alpha_t(x, y) + \varepsilon}{\sum_{k=1}^T (\alpha_k(x, y) + \varepsilon)}, \quad (8)$$

145 with a small $\varepsilon > 0$ to avoid degenerate all-zero cases. In implementation, the weights are treated
 146 as *detached* scalars when multiplying the policy-gradient term, so the update does not introduce
 147 second-order derivatives through the weighting function itself. This keeps the optimization rule close
 148 in spirit and cost to standard critic-free policy gradients. The floor ε is part of the estimator: if all raw
 149 scores vanish relative to this floor, the normalized weights become uniform, while without such a
 150 floor nearly-zero scores can make the normalization ill-conditioned.

Uniform Weighting (Baseline).

$$w_t^{\text{uniform}} = \frac{1}{T}. \quad (9)$$

151 Combined with an RLOO or GRPO baseline, this gives a length-normalized uniform-token baseline
 152 that assigns identical credit to every token in the sampled completion. It has the same token direction
 153 as the unnormalized critic-free token-sum estimator, but differs by the trajectory-length factor $1/T$.

154 **Surprisal Weighting.** Our first intrinsic score is token surprisal,

$$\alpha_t^{\text{surp}}(x, y) = -\log \pi_{\theta}(y_t \mid y_{<t}, x). \quad (10)$$

155 This emphasizes low-probability decisions under the current policy. Intuitively, these are tokens
 156 where the model commits to a less routine continuation and where uniform credit assignment may be
 157 most diluted by predictable filler tokens.

158 **Entropy-Reduction Weighting.** Our second intrinsic score measures local entropy reduction. Let

$$H_t^{\text{pre}}(x, y) = - \sum_v \pi_\theta(v \mid y_{<t}, x) \log \pi_\theta(v \mid y_{<t}, x) \quad (11)$$

159 denote the predictive entropy before sampling token y_t , and let

$$\Delta H_t^{\text{ent}}(x, y) = \max(0, H_t^{\text{pre}}(x, y) - H_{t+1}^{\text{pre}}(x, y)) \quad (12)$$

160 for $t < T$. For the final token we set $\Delta H_T^{\text{ent}} = 0$. We then use

$$\alpha_t^{\text{ent}}(x, y) = \Delta H_t^{\text{ent}}(x, y). \quad (13)$$

161 This score highlights tokens after which the model’s next-step distribution becomes substantially
162 more concentrated. Such events often correspond to commitment points in reasoning, where one
163 branch of continuation becomes much more likely than its alternatives.

164 **Policy-Divergence Weighting.** A third intrinsic score measures how much the current policy has
165 drifted from a reference π_{ref} (typically the base model prior to RL) at each token position:

$$\alpha_t^{\text{div}}(x, y) = |\log \pi_\theta(y_t \mid y_{<t}, x) - \log \pi_{\text{ref}}(y_t \mid y_{<t}, x)|. \quad (14)$$

166 In the RLHF/RLVR setting π_{ref} is already available because it is used to compute the KL regularizer,
167 so this score is essentially free.

168 **Length-Robust Normalization.** Because reasoning trajectories can vary substantially in length,
169 all weights are normalized within each sampled completion rather than across the minibatch. This
170 ensures that longer trajectories do not automatically receive larger aggregate updates simply because
171 they contain more token positions with nonzero salience. The comparison between weighting schemes
172 therefore reflects *where* credit is assigned within a trajectory, not how much total credit a longer
173 sequence receives.

174 **Batch-Level Baselines.** All weighting schemes can be paired with either RLOO or GRPO-style
175 prompt-level baselines. Unless otherwise specified, we use the same rollout groups, optimizer,
176 sampling hyperparameters, and baseline family across methods. This isolates token redistribution as
177 the only algorithmic difference.

178 3.4 Signal Degeneration Under Low-Rank Adaptation

179 So far our exposition has treated the policy π_θ as an unconstrained language model being updated by
180 gradient descent. In practice, however, many LLM-RL pipelines apply LoRA [11], which restricts π_θ
181 to a rank- r perturbation of a frozen reference policy π_{ref} . We now show that the intrinsic weighting
182 schemes defined above are qualitatively degraded by this constraint.

183 We use *collapse* to mean degeneration of the normalized token weights, not only convergence to the
184 perfectly uniform distribution. Depending on the raw score scale, the ε floor, and any sharpening
185 transform, a degraded signal can appear either as uniform broadcast or as spuriously sparse credit
186 concentrated on a tiny set of positions. In both cases, the weights no longer reflect where the adapter
187 has learned to act.

188 **Notation.** Write the logits produced by the base model at position t as $z_t^{\text{ref}} = W_{\text{lm}} h_t^{\text{base}}$ and the
189 logits produced by the adapted model as $z_t^\theta = W_{\text{lm}} h_t^{\text{adapted}}$, where $h_t^{\text{adapted}} = h_t^{\text{base}} + \Delta h_t$ and
190 Δh_t is the sum of LoRA adapter contributions propagated through the transformer to position t . Each
191 adapter contributes $B_\ell A_\ell x_{\ell,t}$ at layer ℓ , where $B_\ell \in \mathbb{R}^{d \times r}$ and $A_\ell \in \mathbb{R}^{r \times d}$, so the raw per-layer
192 perturbation at each position lies in the fixed r -dimensional column space of B_ℓ .

193 **Degeneration of surprisal and entropy.** Because $\|\Delta h_t\|$ is bounded by a product of adapter norms
194 and input activation norms that are themselves roughly stationary across positions in a well-trained
195 base model, the first-order perturbation to per-token surprisal,

$$-\log \pi_\theta(y_t \mid y_{<t}, x) = -\log \pi_{\text{ref}}(y_t \mid y_{<t}, x) + O(\|\Delta h_t\|),$$

196 is dominated by the base model’s surprisal pattern rather than by the LoRA update. The same holds for
197 predictive entropy. Consequently, when we normalize these scores within a trajectory (equation (8)),
198 the resulting weights can be non-uniform, but their non-uniformity is not an adapter-aware credit
199 signal; it mostly reflects base-model uncertainty or the numerics of the normalization.

Degeneration of policy divergence. Divergence weighting, $\alpha_t^{\text{div}} = |\log \pi_\theta(y_t) - \log \pi_{\text{ref}}(y_t)|$, is of direct interest because it quantifies exactly the policy change that the RL update is trying to drive. Under LoRA, however, this quantity is small and approximately uniform across positions for a different reason: the KL budget $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ is bounded by the rank- r adapter’s capacity, so the per-token log-probability ratios are compressed toward zero [2]. Under the within-trajectory normalization in equation (8), this compression is pathological. If the score is approximately constant and the ε floor dominates, the weights approach uniform broadcast. If the floor is negligible, the same tiny differences can be amplified into a spurious sparse distribution.

Consequence. We summarize the practical implication as follows. Let the Gini coefficient $G(w)$ and the effective-token ratio $\text{EffN}(w)/T = 1/(T \sum_t w_t^2)$ be standard concentration measures, with $G(w) = 0$ and $\text{EffN}/T = 1$ corresponding to perfectly uniform weights. For surprisal, entropy-reduction, and divergence weighting under LoRA, these metrics reveal whether the normalized weights have become degenerate: uniform broadcast gives $G(w) \approx 0$, while spuriously sparse credit gives $G(w) \approx 1$ and small EffN/T . Both outcomes are failures of adapter-aware credit assignment. This is *not* a problem with the underlying signals in a fully trainable policy; it is a structural consequence of applying output-distribution signals inside a low-rank adaptation.

3.5 Adapter-Residual Credit Assignment (ARCA)

The degeneration result motivates a different design choice. Rather than measuring per-token properties of the *output distribution* (whose shape is dominated by the frozen base model under LoRA), we measure per-token properties of the *adapter’s own contribution* to the forward pass.

Definition. Given a model with a LoRA adapter, let h_t^{adapted} denote the last-layer hidden state at position t with the adapter enabled, and h_t^{base} the same hidden state with the adapter disabled. Define the Adapter-Residual Credit Assignment salience as

$$\alpha_t^{\text{ARCA}}(x, y) = \|h_t^{\text{adapted}} - h_t^{\text{base}}\|_2, \quad (15)$$

and normalize within the trajectory as in equation (8) to obtain per-token weights w_t^{ARCA} . The adapter residual is computed once per minibatch via an extra no-grad forward pass with the adapter disabled, using `model.disable_adapter()` in frameworks such as PEFT. This is exactly the same call already required to compute a KL regularizer against the reference policy or to support divergence weighting, so ARCA adds no new infrastructure.

Why ARCA avoids output-signal collapse. The key property that distinguishes ARCA from the intrinsic schemes above is that it measures a quantity whose *per-token* value is actively shaped by the attention and MLP pathways through which the adapter input activations are routed. Even for a low-rank adapter with small $\|B_\ell A_\ell\|$, the input activations $x_{\ell,t}$ are non-uniform across positions (attention patterns, layer norms, and content-versus-filler distinctions are already non-uniform in the base model), so $\|\Delta h_t\|$ retains non-trivial position-to-position variation. With a fixed ε floor, any score whose magnitude vanishes completely will eventually become uniform after normalization. ARCA’s claim is therefore not that an infinitesimal adapter can defeat the floor; it is that, while the adapter residual is measurable, its normalized variation is tied to where the adapter actually changes the hidden state rather than to small output-distribution perturbations.

Interpretation as implicit gradient-informed credit. ARCA can be read as a cheap proxy for a gradient-based notion of token importance. Positions at which the adapter contribution is large are positions at which the adapter’s parameter gradient has high contribution to the loss, and therefore positions where a policy-gradient update can have the most effect. This is a substantially weaker statement than the one made by a learned critic, but it has the advantage of being available for free from quantities the adapted forward pass already computes, and of being exactly targeted at the PEFT-specific failure mode introduced above.

4 Experiments

We evaluate whether adapter-residual credit assignment gives a useful token-level signal in the low-rank fine-tuning regime. Because the submission-time experiments are intentionally compact, the

Table 1: **Submission-time sweep.** All runs use Qwen/Qwen3-1.7B-Base, MATH, GRPO, one seed, and 100 training steps. The non-ARCA baselines are matched at LoRA rank 64; ARCA is evaluated at three ranks to separate credit assignment from adapter capacity.

Method	LoRA rank	Trainable params	Role in comparison
Uniform	64	69.7M	Same-rank baseline
Surprisal	64	69.7M	Intrinsic token baseline
Entropy reduction	64	69.7M	Intrinsic token baseline
Policy divergence	64	69.7M	Intrinsic token baseline
ARCA	4	4.36M	Low-rank ARCA control
ARCA	16	17.4M	Mid-rank ARCA control
ARCA	64	69.7M	Same-rank ARCA comparison

empirical section focuses on a single controlled sweep: one model, one task, one advantage estimator, and seven weighting configurations. This design trades breadth for a cleaner test of the central claim: ARCA changes credit assignment without adding trainable parameters beyond the LoRA adapter used by the matched baselines.

4.1 Setup

Model and task. We fine-tune Qwen/Qwen3-1.7B-Base on the MATH training split and evaluate on held-out MATH examples. Prompts ask the model to solve the problem step by step and place the final answer in `\boxed{}`. Rewards are binary exact-match scores computed from the final boxed answer after normalization.

Training. All runs use GRPO prompt-level advantages, LoRA adapters on the attention and MLP projections, and the same rollout budget, optimizer, learning-rate schedule, prompt format, and decoding parameters. Each training step samples $K = 4$ completions per prompt, uses 100 update steps, and evaluates greedy accuracy and pass@4 on 200 held-out MATH examples. We run a single seed, 1337. Checkpoints are saved every 20 steps. We use AdamW with learning rate 5×10^{-6} , zero weight decay, a 10-step warmup followed by cosine decay, batch size 2, gradient accumulation over 4 microbatches, maximum prompt length 512, maximum generation length 512, temperature 1.0, and top- p 0.95 sampling during training.

Methods. The sweep contains seven runs. Four baselines use LoRA rank $r = 64$: uniform token weighting, surprisal weighting, entropy-reduction weighting, and policy-divergence weighting. The proposed method, ARCA, is run at $r \in \{4, 16, 64\}$. The $r = 64$ comparison tests ARCA against baselines with the same trainable parameter count. The $r = 4$ and $r = 16$ comparisons test whether ARCA remains competitive with fewer adapter parameters than the $r = 64$ baselines.

4.2 Main Results on MATH

Table 2 reports held-out MATH performance after RL fine-tuning. The primary comparison is ARCA at rank 64 versus the rank-64 baselines, which holds the trainable parameter count fixed. The rank-4 and rank-16 ARCA runs provide a more stringent control: if lower-rank ARCA is competitive with rank-64 baselines, the gain cannot be attributed simply to using a larger adapter.

Table 2 should be read as a compact validation rather than a broad benchmark. Uniform weighting obtains the highest greedy accuracy, while entropy reduction obtains the highest pass@4. ARCA at rank 64 is competitive with these baselines under the same trainable parameter count, trailing uniform greedy accuracy by three points and exceeding uniform pass@4 by 0.5 points. The lower-rank ARCA variants do not outperform the rank-64 baselines in this run, which is consistent with a capacity-performance tradeoff rather than evidence that ARCA gains come from extra parameters.

4.3 Credit-Assignment Diagnostics

Performance alone does not show whether a method changes token-level credit assignment. We therefore log the concentration of token weights during training. We report the Gini coefficient

Table 2: **Held-out MATH performance.** Greedy accuracy and pass@4 after GRPO fine-tuning. ARCA at rank 64 is parameter-matched to the rank-64 baselines; lower-rank ARCA rows test the parameter-count alternative. Train reward is averaged over the 100 update steps.

Method	LoRA rank	Greedy acc.	Pass@4	Avg. train reward
Uniform	64	0.620	0.675	0.362
Surprisal	64	0.520	0.650	0.326
Entropy reduction	64	0.600	0.700	0.367
Policy divergence	64	0.565	0.670	0.351
ARCA	4	0.525	0.640	0.327
ARCA	16	0.510	0.655	0.336
ARCA	64	0.590	0.680	0.353

Table 3: **Token-weight diagnostics.** Concentration statistics averaged over training. These diagnostics test whether a method produces a non-uniform but non-degenerate token-credit signal, the empirical observable predicted by the degeneration analysis.

Method	LoRA rank	$G(w)$	EffN/T
Uniform	64	0.000	1.000
Surprisal	64	0.941	0.052
Entropy reduction	64	0.920	0.073
Policy divergence	64	0.939	0.055
ARCA	4	0.353	0.467
ARCA	16	0.388	0.460
ARCA	64	0.490	0.334

284 $G(w)$ and effective-token ratio $\text{EffN}(w)/T$. Uniform weighting has $G = 0$ and $\text{EffN}/T = 1$; highly
285 concentrated weights have G near one and a small effective-token ratio.

286 Table 3 is the central diagnostic result. Uniform weighting is exactly flat. Surprisal, entropy reduction,
287 and policy divergence produce extremely concentrated distributions, with effective-token ratios
288 between 0.052 and 0.073 on average. These methods therefore do not collapse to uniform in this
289 implementation; they degenerate in the other direction, into spuriously sparse credit. ARCA occupies
290 the predicted middle regime: it is far from uniform, but it does not concentrate credit onto the tiny
291 effective support used by the output-distribution baselines. This is the empirical signature of the
292 theoretical distinction. Output-distribution scores ask where the model is uncertain or shifted; ARCA
293 asks where the adapter actually acts.

294 4.4 Results and Discussion

295 The results support a theory-first interpretation. The downstream accuracy numbers do not show a
296 universal ARCA win: uniform is best under greedy decoding, and entropy reduction is best under
297 pass@4. However, ARCA at rank 64 is competitive under matched parameters, and its token-
298 credit distribution is qualitatively different from every baseline. In this paper, that diagnostic is
299 not secondary. It is the observable predicted by the LoRA-degeneration analysis: a method whose
300 salience is measured through output distributions can broadcast credit uniformly or concentrate it on
301 a very small set of positions, while adapter-residual salience remains position-discriminative without
302 becoming maximally sparse.

303 This distinction matters because token-level credit assignment is often evaluated only by final task
304 accuracy. The concentration diagnostics separate these cases. ARCA demonstrates that one can
305 obtain an adaptation-aware credit signal from the LoRA adapter itself, with no value head, learned
306 process reward model, or tree construction, while keeping the trainable parameter count identical to
307 the rank-64 baselines.

308 The rank controls provide a useful boundary on the claim. ARCA at ranks 4 and 16 uses substantially
309 fewer trainable parameters than the rank-64 baselines, and in this sweep those lower-rank variants
310 do not match rank-64 performance. We therefore do not claim that ARCA removes the need for

311 sufficient adapter capacity. The supported claim is sharper: at a fixed adapter rank, ARCA changes the
312 geometry of token credit in the way predicted by the theory, and does so while remaining competitive
313 on held-out MATH.

314 4.5 Implementation and Metrics

315 The main experiment entrypoint is `experiment/hf/run_experiment.py`. For each minibatch, the
316 script samples completions from the current policy, computes binary rewards after full generation,
317 constructs GRPO advantages, and applies the weighted token-level objective from Section 3. Surprisal
318 uses current-policy token log probabilities, entropy-reduction uses next-token entropy drops, policy-
319 divergence compares adapted and adapter-disabled log probabilities, and ARCA weights tokens by
320 adapter-residual norms in the final hidden layer.

321 The code records per-step reward mean, reward variance, loss, completion length, token-weight
322 Gini, effective-token ratio, and, for ARCA, the mean and maximum adapter-residual norm. Final
323 evaluation records greedy accuracy and pass@4 on held-out MATH examples. These metrics allow
324 us to distinguish three claims: whether the method trained, whether it changed token-credit structure,
325 and whether that structure improved held-out performance.

326 4.6 Compute

327 Each run fits on a single H100-class GPU. The seven submission-time runs are executed independently,
328 one configuration per GPU, with wall-clock training time on the order of a few hours per run plus final
329 evaluation. The reported sweep therefore requires seven single-GPU runs; preliminary development
330 runs were used to tune runtime and checkpointing but are not included in the reported comparison.

331 4.7 Broader Impacts

332 This work is methodological and studies how to assign token-level credit during reinforcement
333 learning for language models. The potential positive impact is improved sample efficiency and
334 interpretability of RL fine-tuning, especially in settings where practitioners already use parameter-
335 efficient adaptation. The same techniques could also improve the fine-tuning of models for harmful
336 or misleading generation if applied without appropriate task, data, and deployment safeguards. We
337 do not release a new general-purpose model in this paper.

338 4.8 Limitations

339 This sweep is deliberately small: one model, one dataset, one seed, one advantage estimator, and
340 seven configurations. It is sufficient to test the parameter-matched ARCA comparison and the low-
341 rank control, but it does not establish robustness across seeds, tasks, or algorithms. We view the
342 results as a targeted validation of the mechanism rather than a broad benchmark claim.

343 5 Conclusion

344 Token-level credit assignment and parameter-efficient fine-tuning are usually studied as separate
345 design choices. This paper argues that they are coupled. Under LoRA, output-distribution signals
346 such as surprisal, entropy reduction, and policy divergence can degenerate into token weights that
347 no longer track where the adapter acts. ARCA addresses this failure mode by measuring salience
348 where LoRA actually acts: in the adapter-induced hidden-state residual. The resulting signal is
349 lightweight, requires no learned critic or learned process reward model, and produces non-degenerate
350 token-credit distributions in the regime where output-distribution baselines become either flat or
351 highly concentrated. Our MATH/Qwen3-1.7B sweep is intentionally compact, but it supports the
352 central mechanism: adaptation-aware credit assignment changes the geometry of token updates under
353 matched rollout and parameter budgets.

354 References

355 [1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier
356 Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style

- 357 optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual*
358 *Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association
359 for Computational Linguistics, 2024.
- 360 [2] Anonymous. LoRA as an implicit KL regularizer in GRPO fine-tuning: From theory to practice,
361 2026. Under review at Transactions on Machine Learning Research.
- 362 [3] Meng Cao, Shuyuan Zhang, Xiao-Wen Chang, and Doina Precup. SCAR: Shapley credit
363 assignment for more efficient RLHF, 2025.
- 364 [4] Yekun Chai, Haoran Sun, Huang Fang, Shuohuan Wang, Yu Sun, and Hua Wu. MA-RLHF: Re-
365 inforcement learning from human feedback with macro actions. In *The Thirteenth International*
366 *Conference on Learning Representations*, 2025.
- 367 [5] Hongzhan Chen, Tao Yang, Shiping Gao, Ruijun Chen, Xiaojun Quan, Hongtao Tian, and Ting
368 Yao. Discriminative policy optimization for token-level reward models. In Aarti Singh, Maryam
369 Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff,
370 and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*,
371 volume 267 of *Proceedings of Machine Learning Research*, pages 9546–9565. PMLR, 2025.
- 372 [6] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei.
373 Deep reinforcement learning from human preferences. In *Advances in Neural Information*
374 *Processing Systems 30*, 2017.
- 375 [7] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi
376 Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen,
377 Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, Hao Peng, Yu Cheng, Zhiyuan Liu,
378 Maosong Sun, Bowen Zhou, and Ning Ding. Process reinforcement through implicit rewards,
379 2025.
- 380 [8] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement
381 learning, 2025.
- 382 [9] Yuhang He, Haodong Wu, Siyi Liu, Hongyu Ge, Hange Zhou, Keyi Wu, Zhuo Zheng, Qihong
383 Lin, Zixin Zhong, and Yongqi Zhang. Rethinking token-level credit assignment in RLVR: A
384 polarity-entropy analysis, 2026.
- 385 [10] Falko Helm, Nico Daheim, and Iryna Gurevych. Token weighting for long-range language
386 modeling. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for*
387 *Computational Linguistics: NAACL 2025*, pages 1440–1459, Albuquerque, New Mexico, April
388 2025. Association for Computational Linguistics.
- 389 [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang,
390 Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In
391 *International Conference on Learning Representations*, 2022.
- 392 [12] Jian Hu, Jason Klein Liu, Haotian Xu, and Wei Shen. REINFORCE++: Stabilizing critic-free
393 policy optimization with global advantage normalization, 2025.
- 394 [13] Miaobo Hu, BoKun Wang, Shuhao Hu, Ruohan Wang, Xin Wang, Xiaobo Guo, Daren Zha,
395 and Jun Xiao. PSPO: Trainable potential-based reward shaping with internal model signals
396 for post-training policy optimization of large language models, 2026. ICLR 2026 submission,
397 rejected.
- 398 [14] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy,
399 Aaron Courville, and Nicolas Le Roux. VinePPO: Refining credit assignment in RL training of
400 LLMs, 2024.
- 401 [15] Alan Lee and Harry Tong. Token-efficient RL for LLM reasoning, 2025.
- 402 [16] Jiahui Li, Lin Li, Tai-Wei Chang, Kun Kuang, Long Chen, Jun Zhou, and Cheng Yang. Red:
403 Unleashing token-level rewards from holistic feedback via reward redistribution, 2024.

- [17] Shiwei Li, Xiandi Luo, Haozhao Wang, Xing Tang, Ziqiang Cui, Dugang Liu, Yuhua Li, Xiuqiang He, and Ruixuan Li. Beyond higher rank: Token-wise input-output projections for efficient low-rank adaptation, 2025.
- [18] Yingru Li, Jiawei Xu, Ziniu Li, Jiakai Liu, Wei Liu, Yuxuan Tong, Longtao Zheng, Zhenghai Xue, Yaxiang Zhang, Tianle Cai, Ge Zhang, Qian Liu, and Baoxiang Wang. The optimal token baseline: Variance reduction for long-horizon LLM-RL, 2026.
- [19] Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. ReMax: A simple, effective, and efficient reinforcement learning method for aligning large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024.
- [20] Haoming Meng, Kexin Huang, Shaohang Wei, Chiyu Ma, Shuo Yang, Xue Wang, Guoyin Wang, Bolin Ding, and Jingren Zhou. Sparse but critical: A token-level analysis of distributional shifts in RLVR fine-tuning of LLMs, 2026.
- [21] Julian Minder, Clément Dumas, Stewart Slocum, Helena Casademunt, Cameron Holmes, Robert West, and Neel Nanda. Narrow fine-tuning leaves clearly readable traces in activation differences, 2025.
- [22] Youssef Mroueh, Nicolas Dupuis, Brian Belgodere, Apoorva Nitsure, Mattia Rigotti, Kristjan Greenewald, Jiri Navratil, Jerret Ross, and Jesus Rios. Revisiting group relative policy optimization: Insights into on-policy and off-policy training, 2025.
- [23] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35*, 2022.
- [24] Prasanna Parthasarathi, Mathieu Reymond, Boxing Chen, Yufei Cui, and Sarath Chandar. Grpo- λ : Credit assignment improves llm reasoning, 2025.
- [25] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897, Lille, France, 07–09 Jul 2015. PMLR.
- [26] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.
- [27] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [28] Zikang Shan, Han Zhong, Liwei Wang, and Li Zhao. Bringing value models back: Generative critics for value modeling in LLM reinforcement learning, 2026.
- [29] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.
- [30] Lloyd S. Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker, editors, *Contributions to the Theory of Games II*, pages 307–317. Princeton University Press, Princeton, 1953.
- [31] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems 33*, 2020.
- [32] Hieu Tran, Zonghai Yao, and Hong Yu. Exploiting tree structure for credit assignment in RL training of LLMs, 2025.

- 452 [33] Shangshang Wang, Julian Asilis, Ömer Faruk Akgül, Enes Burak Bilgin, Ollie Liu, and Willie
453 Neiswanger. Tina: Tiny reasoning models via LoRA, 2025.
- 454 [34] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui
455 Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji
456 Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority
457 tokens drive effective reinforcement learning for LLM reasoning, 2025.
- 458 [35] Xumeng Wen, Zihan Liu, Shun Zheng, Shengyu Ye, Zhirong Wu, Yang Wang, Zhijian Xu,
459 Xiao Liang, Junjie Li, Ziming Miao, Jiang Bian, and Mao Yang. Reinforcement learning with
460 verifiable rewards implicitly incentivizes correct reasoning in base LLMs, 2025.
- 461 [36] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement
462 learning. *Machine Learning*, 8:229–256, may 1992.
- 463 [37] Wei Xiong, Jiarui Yao, Yuhui Xu, Bo Pang, Lei Wang, Doyen Sahoo, Junnan Li, Nan Jiang,
464 Tong Zhang, Caiming Xiong, and Hanze Dong. A minimalist approach to LLM reasoning: from
465 rejection sampling to reinforce, 2025.
- 466 [38] Shentao Yang, Shujian Zhang, Congying Xia, Yihao Feng, Caiming Xiong, and Mingyuan Zhou.
467 Preference-grounded token-level guidance for language model fine-tuning. In Alice Oh, Tristan
468 Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances
469 in Neural Information Processing Systems 36: Annual Conference on Neural Information
470 Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023*, 2023.
- 471 [39] Qingyu Yin, Yulun Wu, Zhennan Shen, Sunbowen Li, Zhilin Wang, Yanshu Li, Chak Tou
472 Leong, Jiale Kang, and Jinjin Gu. Evaluating parameter efficient methods for RLVR, 2025.
- 473 [40] Hee Suk Yoon, Eunseop Yoon, Mark A. Hasegawa-Johnson, Sungwoong Kim, and Chang D.
474 Yoo. ConfPO: Exploiting policy model confidence for critical token selection in preference opti-
475 mization. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp,
476 Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International
477 Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*,
478 pages 72641–72655. PMLR, 2025.
- 479 [41] Song Yu, Li Li, Wenwen Zhao, and Zhisheng Yang. ERPO: Token-level entropy-regulated
480 policy optimization for large reasoning models, 2026.
- 481 [42] Chong Zhang, Yue Deng, Xiang Lin, Bin Wang, Dianwen Ng, Hai Ye, Xingxuan Li, Yao Xiao,
482 Zhanfeng Mo, Qi Zhang, and Lidong Bing. 100 days after deepseek-r1: A survey on replication
483 studies and more directions for reasoning language models, 2025.
- 484 [43] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang,
485 Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy
486 optimization, 2025.
- 487 [44] Han Zhong, Zikang Shan, Guhao Feng, Wei Xiong, Xinle Cheng, Li Zhao, Di He, Jiang Bian,
488 and Liwei Wang. DPO meets PPO: Reinforced token optimization for RLHF. In *Proceedings
489 of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of
490 Machine Learning Research*, pages 78498–78521. PMLR, 2025.

A Extended Related Work

A.1 From RLHF to RLVR

Modern language-model post-training inherits its optimization machinery from policy-gradient reinforcement learning, from REINFORCE [36] to trust-region and clipped-surrogate methods such as TRPO and PPO [25, 27]. Advantage estimation, especially GAE, became the standard variance-reduction device in continuous-control RL by learning value functions over states or prefixes [26]. In language modeling, these ideas entered mainstream use through RLHF systems that optimize sequence-level rewards derived from human preferences or reward models [6, 31, 23].

The recent reasoning-model wave shifted part of the field from preference-based RLHF toward reinforcement learning with verifiable rewards (RLVR), where supervision is often a deterministic correctness signal on math or code tasks. DeepSeekMath introduced GRPO as a practical critic-free alternative for these settings [29], and DeepSeek-R1 popularized large-scale RL-only or RL-dominant reasoning pipelines [8]. Subsequent surveys document how quickly this regime became the default template for open reasoning-model replication and extension [42]. This transition matters for our setting because RLVR makes sparse outcome rewards and long reasoning trajectories central, thereby exposing the credit-assignment problem more directly than earlier short-form RLHF tasks.

A.2 Critic-Based and Critic-Free Policy Optimization

The classical solution to delayed reward is to learn a critic or value function and convert returns into token- or prefix-level advantages. PPO-style RLHF pipelines follow this recipe, but the value-estimation problem is particularly awkward for text generation because prefixes are high-dimensional, non-Markov, and semantically heterogeneous. VinePPO demonstrated that standard value heads produce poor estimates of expected returns for reasoning tasks and proposed Monte Carlo vine-style rollout estimates as a more accurate alternative [14]. GenAC extends this idea by replacing scalar value prediction with a generative critic that uses chain-of-thought reasoning for value estimation [28].

A growing body of work questions whether value heads are necessary at all in LLM post-training. ReMax replaces learned critics with a greedy baseline and shows that much of PPO’s practical benefit can be retained with a simpler REINFORCE-style objective [19]. Back to Basics systematically compares PPO with RLOO-style estimators and finds that critic-free methods can match or exceed value-based baselines in RLHF [1]. REINFORCE++ further strengthens this line by replacing the prompt-level advantage normalization used by GRPO and RLOO with a global, batch-level normalization, stabilizing critic-free policy optimization without reintroducing a learned value function [12]. In reasoning-focused RLVR, GRPO normalizes rewards across a group of sampled responses rather than learning prefix values [29], and later work refines this family with theoretical analyses, off-policy variants, and sequence-level formulations such as GSPO [22, 43]. These methods substantially simplify the optimization stack, but they generally still broadcast a trajectory-level signal across all tokens once a scalar baseline is fixed.

A.3 Reasoning-Specific Analyses of RLVR

Another recent line asks why critic-free RLVR works so well for reasoning in the first place. A Minimalist Approach to LLM Reasoning shows that a simple rejection-sampling baseline (RAFT) is surprisingly competitive with GRPO and PPO on reasoning tasks, and that GRPO’s advantage over vanilla REINFORCE comes primarily from discarding prompts whose sampled responses are all incorrect rather than from reward normalization; the authors distill this insight into Reinforce-Rej, a minimal REINFORCE variant that filters both entirely-correct and entirely-incorrect samples, suggesting that much of the field’s complexity is optional rather than essential [37]. Complementarily, Wen et al. analyze RLVR theoretically and empirically, arguing that answer-level verifiable rewards can nonetheless incentivize correct intermediate reasoning early in a trajectory [35]. These results are important because they show that uniform outcome-level objectives already contain nontrivial reasoning signal.

At the same time, they do not eliminate the question of *which* tokens should absorb that signal. Recent empirical analysis of RLVR training dynamics suggests that improvements are disproportionately driven by a minority of high-entropy “forking” tokens, and that restricting updates to this subset can

remain competitive or even outperform full-token updates in some settings [34]. Building on this observation, EAPO adapts conditional mutual information to the autoregressive RLVR setting and proves that the credit a token can carry is upper-bounded by its entropy, motivating an entropy-aware modulation of per-token learning signals [9]. ERPO similarly identifies “critical decision pivots” at high-entropy states and applies targeted exploration at those positions [41]. *Sparse but Critical* argues, via a distributional-shift analysis based on cross-sampling between policy and reference, that a small fraction of tokens accounts for most of the useful update magnitude in RLVR, and proposes a divergence-based selection rule for identifying them [20]. These entropy- and divergence-based methods are the closest concurrent work to the intrinsic weighting schemes that form our baselines; the present paper does *not* claim priority over them. Instead, our contribution is to show that their underlying signals are structurally degraded under LoRA, and to provide a credit-assignment mechanism (ARCA) that does not suffer the same degradation.

A.4 Fine-Grained Credit Assignment Beyond Uniform Token Updates

A large parallel literature attempts to make supervision denser than a single end-of-trajectory reward. One family learns explicit token- or process-level reward estimators. Preference-grounded Token-level Guidance derives token-level signals from preference data for fine-tuning [38], while Discriminative Policy Optimization learns token-level Q-style reward models from preferences without requiring fine-grained annotation [5]. PRIME pushes this direction further for reasoning tasks by constructing implicit process rewards and updating process reward models online from rollouts and outcome labels [7]. Reinforced Token Optimization (RTO) frames RLHF as a token-level MDP and uses DPO log-likelihood ratios as a per-token reward signal derived from the preference model, which is then optimized with PPO to produce fine-grained token-level credit [44]. These methods can produce rich token-level feedback, but they rely on an auxiliary reward-modeling component that our approach intentionally avoids.

A second family learns a small token-level critic on top of the policy’s own hidden states (rather than an external reward model) and uses its predictions to form token-level advantages. This corresponds closely to an earlier version of our own framework, and we found through an extensive literature review that the token-level actor–critic design space was already well-covered: VinePPO [14] and GenAC [28] cover non-parametric and generative critics respectively, and OTB [18] provides a principled variance-minimizing position-dependent baseline. We therefore do *not* position a critic-based configuration as the proposed method, and the submission-time sweep focuses on critic-free intrinsic weighting schemes.

A third family redistributes outcome rewards algorithmically rather than by training a dense reward model. RED derives token-level rewards by redistributing holistic reward-model scores over a trajectory [16]. SCAR uses Shapley-inspired attribution to estimate token or span contributions from sequence-level feedback [3, 30]. TEMPO uses groups of sampled solutions to build a prefix tree and compute nonparametric prefix values, yielding branch-sensitive corrections without a learned critic [32]. GRPO- λ introduces eligibility traces and lambda-returns into critic-free RLVR to propagate outcome information backward along the sequence [24]. OTB derives an optimal position-dependent baseline that minimizes per-token gradient variance; the practical estimator sets the baseline at position t to a cumulative-gradient-energy-weighted average of group returns-to-go, where the weights are a causal logit-gradient proxy computed directly from the forward-pass probabilities [18]. PSPO applies potential-based reward shaping theory to construct dense token rewards from outcome signals without altering the optimal policy [13]. Compared with these approaches, our focus is orthogonal: we do not propose a new redistribution rule, but instead diagnose a failure mode that can affect intrinsic-signal-based redistribution methods when applied inside LoRA.

A.5 Alternative Action Granularities and Token Selection

Several papers attack the same underlying problem by changing the optimization unit rather than by redesigning the reward. MA-RLHF introduces macro actions, grouping tokens into higher-level units to shorten the temporal distance between decisions and rewards [4]. GSPO moves importance weighting and clipping from the token level to the sequence level for greater stability in large-scale RL training [43]. These methods reinforce the broader point that the granularity of optimization matters as much as the reward definition.

Outside on-policy RL proper, token-selective optimization has also been explored in adjacent paradigms. ConfPO emphasizes preference-critical tokens based on policy confidence in preference optimization [40], and token weighting for long-range language modeling shows that non-uniform token emphasis can improve optimization even in supervised settings [10]. Together with high-entropy token analyses in RLVR [34], these results strongly suggest that uniform token treatment is an arbitrary design choice rather than a necessity.

B Additional Method Context

B.1 Relationship to Existing Methods

Our formulation recovers standard critic-free RL as a special case: uniform weighting plus a prompt-level batch baseline yields the usual RLOO/GRPO-style estimator. The intrinsic weighting schemes (surprisal, entropy reduction, policy divergence) are the natural baselines within our framework and map cleanly to published methods (e.g., divergence weighting corresponds to the divergence-advantage analyses of *Sparse but Critical* [20], entropy-based weighting to [9, 41]). ARCA is the only scheme we study that uses an explicitly *adaptation-aware* signal. Unlike critic-based PPO, ARCA does not learn a value head. Unlike RED, PRIME, or token-level reward-model methods, ARCA does not construct dense rewards from an auxiliary model. Unlike TEMPO or related tree-based methods, ARCA does not build prefix graphs or estimate branch values. Unlike hard token-selection methods based on top-entropy masking, ARCA retains a dense token update but modulates it continuously using an intrinsic salience score derived from the adapter itself.

The resulting estimator is intentionally minimal. It changes only the within-trajectory allocation of a scalar on-policy advantage, using quantities already available from the forward pass with the adapter disabled. This makes it easy to layer on top of existing critic-free RLVR pipelines while preserving their computational simplicity.

C Theoretical Interpretation

We provide an interpretation of intrinsic token weighting as a variance-control and credit-redistribution mechanism for policy gradient estimation in language models. Our goal is not to derive new convergence guarantees, but to clarify how token weighting relates to advantage estimation and why it can be effective without learning value functions.

C.1 Token Weighting as Credit Redistribution

Consider the standard REINFORCE estimator with a batch-level baseline:

$$g = (R - b) \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(y_t \mid y_{<t}, x). \quad (16)$$

This estimator assigns equal credit to all tokens in a trajectory. Introducing token weights yields:

$$g_w = (R - b) \sum_{t=1}^T w_t(y) \nabla_{\theta} \log \pi_{\theta}(y_t \mid y_{<t}, x), \quad (17)$$

where $\sum_t w_t = 1$.

Importantly, this transformation does not change the reward signal itself; it redistributes how the trajectory-level signal is attributed to individual decisions. When w_t depends only on quantities available at sampling time (e.g., log-probabilities or entropy), the estimator remains on-policy and does not require learning additional functions.

C.2 Relationship to Advantage Estimation

Advantage-based methods can be interpreted as implicitly defining token-level weights via a learned value function:

$$A_t = R - V(y_{<t}, x). \quad (18)$$

635 In this view, the contribution of each token is scaled according to how much the observed outcome
636 deviates from an estimated expectation conditioned on the prefix.

637 However, this interpretation relies on the existence of a meaningful value function over prefixes. In
638 language generation, prefixes are non-Markov and do not form a stable state space, making $V(y_{<t}, x)$
639 difficult to estimate and prone to bias. Token weighting offers an alternative: rather than estimating
640 expected returns from prefixes, it emphasizes tokens based on intrinsic measures of decision salience,
641 such as uncertainty or information gain.

642 C.3 Connection to Control Variates

643 From a statistical perspective, subtracting a baseline b is a form of control variate that reduces variance
644 without affecting unbiasedness. Token weighting can be viewed as a complementary mechanism that
645 reshapes the contribution of individual score-function terms:

$$\nabla_{\theta} \log \pi_{\theta}(y) = \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(y_t \mid y_{<t}, x). \quad (19)$$

646 Uniform weighting treats all score-function terms equally, regardless of their variance or relevance.
647 Intrinsic weighting schemes effectively reweight these terms, down-weighting low-variance or low-
648 impact contributions (e.g., high-probability continuation tokens) and emphasizing high-variance or
649 high-impact decisions (e.g., low-probability or uncertainty-reducing tokens). While this reweighting
650 introduces bias relative to the uniform estimator, it can substantially reduce variance, leading to
651 improved optimization dynamics in practice.

652 C.4 Interpretation of Specific Weighting Schemes

653 **Surprisal Weighting.** Surprisal-based weights emphasize tokens with low conditional probability
654 under the current policy. These tokens correspond to decisions where small parameter changes can
655 induce large changes in likelihood, and therefore often dominate gradient variance. Weighting by
656 surprisal concentrates learning signal on such high-sensitivity decisions.

657 **Entropy-Reduction Weighting.** Entropy-reduction weighting emphasizes tokens that sharply
658 reduce predictive entropy. These tokens correspond to commitment points in generation, where the
659 model transitions from a diffuse distribution over continuations to a more concentrated one. This
660 mirrors the role of branching points in tree-based reasoning methods, but is computed locally from
661 the model’s predictive distribution without explicit tree construction.

662 C.5 Bias–Variance Tradeoff

663 Both advantage estimation and intrinsic token weighting introduce bias in exchange for variance
664 reduction. Advantage estimation does so by relying on a learned value function, whose bias can be
665 substantial when the underlying state abstraction is weak. Intrinsic token weighting introduces bias
666 by reshaping the contribution of token-level gradients based on heuristic salience measures. The
667 empirical question is therefore not whether bias is introduced, but whether the bias–variance tradeoff
668 is favorable.

669 This motivates the empirical comparison implemented in the repository: if intrinsic token weighting
670 improves accuracy, $\text{pass}@k$, or optimization behavior under matched settings, then the bias it
671 introduces may be more benign than the bias induced by ill-defined value targets over text prefixes.
672 The point is not that weighting is unbiased, but that its inductive bias may be better aligned with
673 token-level credit assignment in language generation.

674 C.6 Signal Degeneration Under Low-Rank Adaptation

675 The preceding analysis assumes that the intrinsic weights $\{w_t\}_{t=1}^T$ are non-degenerate: if every
676 scheme becomes uniform or spuriously sparse for reasons unrelated to the learned adapter, then token
677 weighting cannot provide meaningful credit redistribution over a uniform baseline. We now formalize
678 when this degeneracy occurs under LoRA.

Setup. Let π_{ref} be a frozen base model with last-layer hidden states $h_t^{\text{base}} \in \mathbb{R}^d$ and a LoRA adapter whose net contribution at position t is $\Delta h_t \in \mathbb{R}^d$, giving adapted hidden states $h_t^{\text{adapted}} = h_t^{\text{base}} + \Delta h_t$ and logits $z_t^\theta = W_{\text{lm}} h_t^{\text{adapted}}$. Let $\beta = \max_t \|\Delta h_t\|_2$ denote the maximum per-position adapter contribution and $L = \|W_{\text{lm}}\|_{\text{op}}$ its logit-space Lipschitz constant. Write $G(\cdot)$ for the Gini coefficient of a non-negative vector and $\text{EffN}(w)/T = 1/(T \sum_t w_t^2)$ for the normalized effective-token count. When a normalization uses an ε floor, the statements below are interpreted in the regime where the raw salience scores dominate ε ; if fixed ε dominates all scores, every scheme reverts to uniform by construction.

Proposition 1 (Degeneration of surprisal and entropy weighting). *Let $\alpha_t^{\text{surp}} = -\log \pi_\theta(y_t \mid y_{<t}, x)$ and $\alpha_t^{\text{surp,ref}} = -\log \pi_{\text{ref}}(y_t \mid y_{<t}, x)$, with corresponding normalized weights w^{surp} and $w^{\text{surp,ref}}$. Then for every trajectory,*

$$\|w^{\text{surp}} - w^{\text{surp,ref}}\|_1 \leq \frac{4L\beta}{\sum_t \alpha_t^{\text{surp,ref}}}.$$

In particular, the adapted surprisal weights remain close to the reference model’s surprisal profile rather than becoming an adapter-specific credit signal. If that reference profile is close to uniform, the weights approach $1/T$; if it is highly uneven, the weights can instead remain spuriously concentrated for reasons inherited from the base model. The analogous statement holds for the entropy-reduction score.

Proof sketch. The softmax Jacobian is 2-Lipschitz in the log-space norm, so the per-token log-probabilities differ by at most $2L\beta$ between the adapted and base model. The projection onto the simplex via within-trajectory normalization is 1-Lipschitz in ℓ_1 up to a factor of $2/\sum_t \alpha_t^{\text{ref}}$; combining these gives the stated bound. $\square$

Proposition 2 (Degeneration of divergence weighting). *Let $\alpha_t^{\text{div}} = |\log \pi_\theta(y_t \mid y_{<t}, x) - \log \pi_{\text{ref}}(y_t \mid y_{<t}, x)|$ and $w^{\text{div}} = (\alpha^{\text{div}} + \varepsilon) / \sum_t (\alpha_t^{\text{div}} + \varepsilon)$. Then*

$$0 \leq \alpha_t^{\text{div}} \leq 2L\beta \quad \forall t,$$

so if ε is fixed and $\beta \rightarrow 0$, then $w^{\text{div}} \rightarrow 1/T$. If $\varepsilon = 0$ or is negligible relative to the raw scores, the same normalization is ill-conditioned as $\beta \rightarrow 0$: the limiting weights are determined by tiny relative differences among vanishing log-probability shifts rather than by a robust adapter-credit signal.

The key observation is that the intrinsic scores are all determined by the per-token *log-probability differential*, which is precisely what LoRA’s rank- r constraint drives small and approximately uniform across positions. The degeneration is not a property of a particular weighting scheme; it is a property of measuring per-token variation through the output distribution when the policy has been constrained to a small neighborhood of the reference.

C.7 ARCA and Position-Discriminative Signal

ARCA side-steps this degeneration by measuring a quantity that is position-varying even when the adapter’s logit-space impact is small.

Proposition 3 (ARCA remains non-degenerate). *Suppose the adapter-induced residual admits the decomposition $\Delta h_t = \sum_\ell B_\ell A_\ell x_{\ell,t}^{\text{pre}} + e_t$, where $x_{\ell,t}^{\text{pre}}$ is the pre-adapter activation at layer ℓ and position t and $\|e_t\|$ is a higher-order cross-layer term. Let $v_{\ell,t} = B_\ell A_\ell x_{\ell,t}^{\text{pre}}$ and define $V_\ell = \text{Var}_t(\|v_{\ell,t}\|)$ and $\bar{V}_\ell = \mathbb{E}_t[\|v_{\ell,t}\|]$. Then*

$$\text{Var}_t(\alpha_t^{\text{ARCA}}) \geq \max_\ell V_\ell - O\left(\sum_\ell \bar{V}_\ell \|e_t\|\right).$$

Assume the residual scores dominate the normalization floor ε . In particular, as long as the pre-adapter activations $x_{\ell,t}^{\text{pre}}$ have non-trivial position-to-position variance at any layer, α_t^{ARCA} has bounded-below variance across positions, and its normalized weights w_t^{ARCA} do not converge to the uniform distribution merely because the output-distribution shift is small. With fixed ε , however, all scores become uniform in the limit where the adapter residual itself vanishes below the floor.

The intuitive content of Proposition 3 is that ARCA inherits its position-discrimination from the base model’s own activation pattern (which is already non-uniform because attention and layer norm

723 routing produce content-versus-filler distinctions) rather than from a quantity that LoRA is specifically
724 constrained to make small. Scaling the adapter weights uniformly scales α^{ARCA} uniformly, so the
725 normalized weights w^{ARCA} are scale-invariant in the adapter magnitude.

726 **C.8 Bias–Variance Tradeoff**

727 Both advantage estimation and intrinsic token weighting introduce bias in exchange for variance
728 reduction. Advantage estimation does so by relying on a learned value function, whose bias can be
729 substantial when the underlying state abstraction is weak. Intrinsic token weighting introduces bias
730 by reshaping the contribution of token-level gradients based on heuristic salience measures. The
731 empirical question is therefore not whether bias is introduced, but whether the bias–variance tradeoff
732 is favorable—and, per Propositions 1 and 2, whether the bias-inducing signal survives the LoRA
733 bottleneck at all.

734 **C.9 Summary**

735 This perspective reframes advantage estimation as one particular approach to credit redistribution in
736 policy gradient methods. Intrinsic token weighting offers an alternative that leverages the structure
737 of the language model’s predictive distribution, without assuming the existence of meaningful state
738 values. Under LoRA, however, the output-distribution-based signals that most published weighting
739 methods rely on can degenerate into uniform or spuriously sparse token weights; the adapter-residual
740 signal that ARCA uses is, by Proposition 3, non-degenerate whenever the adapter residual remains
741 measurable above the normalization floor.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes].

Justification: The abstract and introduction frame ARCA as a mechanism for testing whether adapter-residual token credit remains non-degenerate under LoRA, and the experiments section limits the empirical scope to the seven-run MATH/Qwen3-1.7B sweep.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes].

Justification: Section 4 includes a limitations subsection stating that the current sweep uses one model, one dataset, one seed, one advantage estimator, and seven configurations.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [No].

Justification: Section C states the assumptions and gives proof sketches for the collapse and non-degeneracy propositions, but the current submission does not include full formal proofs for every theoretical statement.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Section 4 specifies the model, task, reward, advantage estimator, LoRA ranks, rollout budget, evaluation size, seed, optimizer, and decoding hyperparameters used for the reported seven-run sweep.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No].

Justification: The current submission does not include an anonymized public code or data release. The paper nevertheless reports the model, task, seed, hyperparameters, reward construction, evaluation protocol, and compute needed to reproduce the main comparison.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: Section 4 describes the MATH train/test setting, prompt format, reward, optimizer, learning-rate schedule, LoRA ranks, rollout count, decoding parameters, checkpointing, and final evaluation metrics.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No].

Justification: The submission-time sweep uses a single seed and does not report confidence intervals or significance tests. The limitations subsection explicitly notes that the results do not establish robustness across seeds, tasks, or algorithms.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes].

Justification: Section 4 states that each run fits on a single H100-class GPU, that the seven reported configurations are independent single-GPU runs, and that preliminary development runs are not part of the reported comparison.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes].

Justification: The work is a methodological study of language-model reinforcement learning, uses public model and benchmark assets, does not involve human-subject data collection, and discusses potential broader impacts and misuse considerations.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes].

Justification: Section 4 includes a broader impacts subsection discussing potential positive effects on sample efficiency and interpretability, as well as potential misuse in harmful or misleading fine-tuning settings.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A].

Justification: The paper does not release a new high-risk general-purpose pretrained model or scraped dataset; the reported work uses existing public model and dataset assets for a methodological study.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No].

Justification: The paper credits the public model and benchmark families used in the experiments, but the current manuscript does not explicitly enumerate the licenses and terms of use for every existing software, model, and dataset asset.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A].

Justification: The submission does not release a new dataset, model, benchmark, or other standalone asset.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A].

Justification: The paper does not involve crowdsourcing experiments or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

1060 Answer: [N/A].

1061 Justification: The paper does not involve crowdsourcing experiments or research with human

1062 subjects, so IRB or equivalent human-subjects review is not applicable.

1063 Guidelines:

- 1064 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
- 1065 with human subjects.
- 1066 • Depending on the country in which research is conducted, IRB approval (or equivalent)
- 1067 may be required for any human subjects research. If you obtained IRB approval, you
- 1068 should clearly state this in the paper.
- 1069 • We recognize that the procedures for this may vary significantly between institutions
- 1070 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
- 1071 guidelines for their institution.
- 1072 • For initial submissions, do not include any information that would break anonymity (if
- 1073 applicable), such as the institution conducting the review.

1074 **16. Declaration of LLM usage**

1075 Question: Does the paper describe the usage of LLMs if it is an important, original, or

1076 non-standard component of the core methods in this research? Note that if the LLM is used

1077 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

1078 scientific rigor, or originality of the research, declaration is not required.

1079 Answer: [Yes].

1080 Justification: Large language models are central to the research object and methodology;

1081 the paper identifies the model family, LoRA fine-tuning setting, reward construction, and

1082 generation/evaluation procedure in Section 4.

1083 Guidelines:

- 1084 • The answer [N/A] means that the core method development in this research does not
- 1085 involve LLMs as any important, original, or non-standard components.
- 1086 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
- 1087 be described.